Nebula Ledger: Code Report

A review of index.html, style.css and script.js

1. Summary

Nebula Ledger is a single-page bill splitter built with plain HTML, CSS and JavaScript, with no frameworks and no build step. Roughly 750 lines across three files (105 HTML, 142 CSS, 502 JavaScript) deliver a complete app: people can be added, expenses split four ways, debts simplified into a short payment plan, and every action is recorded in a hash-chained ledger that exposes tampering. A canvas view shows the group as a constellation of planets and comets.

The code is well organised and careful about correctness, particularly around money and escaping user text. The main findings are small: a file-name mismatch, a slightly overstated security claim, and a few polish items (Section 6).

2. Architecture

The project follows a clear three-layer split.

- **index.html** defines the structure: a header and a main grid of five panels (Crew, Stage, Expense, Plan, Chain).
- **style.css** defines the presentation: design tokens, grid layout, components and responsive rules.
- **script.js** defines the behaviour and is itself divided into three parts: Engine (pure logic, no DOM), UI (forms, lists, event wiring) and Cosmos (canvas renderer).

Data flows in one direction. A user action calls Engine.append, then refresh() rebuilds the derived state with Engine.state() and Engine.verify(), re-renders every panel, and finally tells Cosmos to redraw.

3. index.html

The markup is compact and semantic.

- **Landmarks:** a header, a main element and one section per panel. Each section carries a class (crew, stage, expense, plan, chain) that CSS maps to a grid area.

- **Controls:** every input has a label or an aria-label. The split-mode selector is a group of buttons using aria-pressed. The toast uses role="status" so changes are announced.
- **Canvas:** the cosmos canvas has an aria-label describing it; a tooltip div and a legend are overlaid on top.
- **Import:** a hidden file input is triggered by the visible Import button.
- **Loading:** Google Fonts are preconnected, the stylesheet is linked in the head and the script is loaded at the end of the body.

4. style.css

4.1 Design tokens

All colours, radii and font stacks are defined as custom properties on :root. The palette is deliberately meaningful: cyan (--owed) marks someone who is owed money, orange (--owes) marks someone who owes, green means settled and pink means error. Three radii (22, 14 and 9px) keep panels, rows and buttons consistent. Two typefaces are used: Bricolage Grotesque for text and IBM Plex Mono for money and hashes.

4.2 Layout and responsiveness

The page uses CSS Grid with named areas. On desktop there are three columns (crew and expense on the left, stage and plan in the middle, chain on the right spanning both rows). At 1180px and below the layout becomes two columns with the stage across the top, and at 720px and below it becomes a single stacked column. Because areas are named per breakpoint, the HTML never changes. The chain panel is position: sticky on desktop and static below 1180px.

4.3 Components

- A single .btn base class with modifiers: primary, warm, ghost, tiny and danger.
- A four-column segmented control whose active state is driven by aria-pressed.
- A chain timeline built from a left border and a ::before dot, coloured by entry type, with a red state for entries after a broken link.
- Stat cards floating over the canvas with a blurred glass backdrop.
- A fixed toast that slides and fades using a CSS transition.

4.4 Accessibility

The stylesheet sets color-scheme: dark, draws a visible :focus-visible outline, and disables transitions when prefers-reduced-motion is requested.

5. script.js

5.1 Engine

Hash-chained ledger. Each entry has the shape {i, t, type, data, prev, hash}. The hash is the SHA-256 of the other five fields, and prev stores the previous entry's hash (or GENESIS for the first). append() is the only way to write. verify() walks the chain and returns the index of the first entry whose link, index or hash is wrong, or -1 if the chain is intact. Importing a file runs verify() first and rejects it on failure. Mistakes are handled by adding a void entry that points at the original, never by deleting.

Integer cents. All amounts are stored as whole cents, which removes floating-point error.

Splitting. proportional() uses the largest-remainder method: it floors each share, then gives the leftover cents to the largest fractional parts, so shares always add up exactly to the bill. For example, 100.00 split three ways gives 33.34, 33.33 and 33.33. computeShares() supports four modes: equal, exact (amounts must sum to the bill), percent (must total 100%) and shares (weighted).

State by replay. state() replays the chain from the start, skipping voided entries, to compute people, balances and the total. It also counts how many pairwise debts would exist without simplification, which feeds the "transfers saved" statistic.

Settle-up plan. simplify() repeatedly pairs the largest debtor with the largest creditor and transfers the smaller of the two amounts until every balance is zero.

5.2 UI

Every action ends in refresh(), which recomputes state, verifies the chain and re-renders the badge, crew list, payer menu, split rows, plan, chain and statistics. A few details are worth noting:

- esc() escapes every user-supplied string before it is placed into innerHTML.
- renderSplitRows() builds a signature from the mode and the people list and only rebuilds the form when it changes, so typed values survive other updates.

- Validation errors from the Engine are shown in the hint line under the split rows; success and information use a toast.
- Export downloads the ledger as JSON. A "Try to tamper" button edits an old entry in memory only, so the chain's detection can be demonstrated, and "Restore from storage" reloads the saved copy.

5.3 Cosmos canvas

A requestAnimationFrame loop redraws the scene each frame. Each person is a planet whose radius scales with the size of their balance, with a ring coloured by status. Payments in the plan are drawn as curved comets from debtor to creditor, with seven moving particles and an amount label. A field of 160 twinkling stars sits behind them. The renderer caps the device pixel ratio at 2, honours prefers-reduced-motion, and uses a hover hit-test to drive the tooltip.

6. Findings and recommendations

1. **Script file name.** index.html loads script.js, but the uploaded file is script__4_.js. Rename it or update the script tag, otherwise the page will not run.
2. **Tamper-evident, not tamper-proof.** The page title and tagline call the ledger tamper-proof. Because the chain and its storage are both client-side, anyone who can write to localStorage can recompute every hash. Digital signatures or anchoring the latest hash somewhere external would be needed for a stronger guarantee; otherwise "tamper-evident" is the more accurate wording.
3. **"Fewest payments" claim.** The Plan panel says it shows the fewest payments that clear every balance. The greedy matching is fast and good for small groups, but it is not guaranteed to be minimal, since the true minimum is an NP-hard problem.
4. **Fallback hash.** When crypto.subtle is unavailable (for example on a non-secure origin), the code falls back to cyrb53. This is not cryptographic and produces a shorter hash, so a warning would be sensible.
5. **Currency formatting.** money() always formats numbers with the en-IN locale, whatever currency symbol the user types.
6. **Small items.** The Load demo button calls loadDemo() without awaiting it, and the canvas could carry role="img" alongside its aria-label.

7. Conclusion

Nebula Ledger is a tidy, dependency-free project with a strong separation between logic, interface and rendering. Its handling of money, escaping and accessibility is better than most projects of this size. Fixing the file-name mismatch is the only change needed to run it as supplied; the remaining items are refinements to wording and polish.